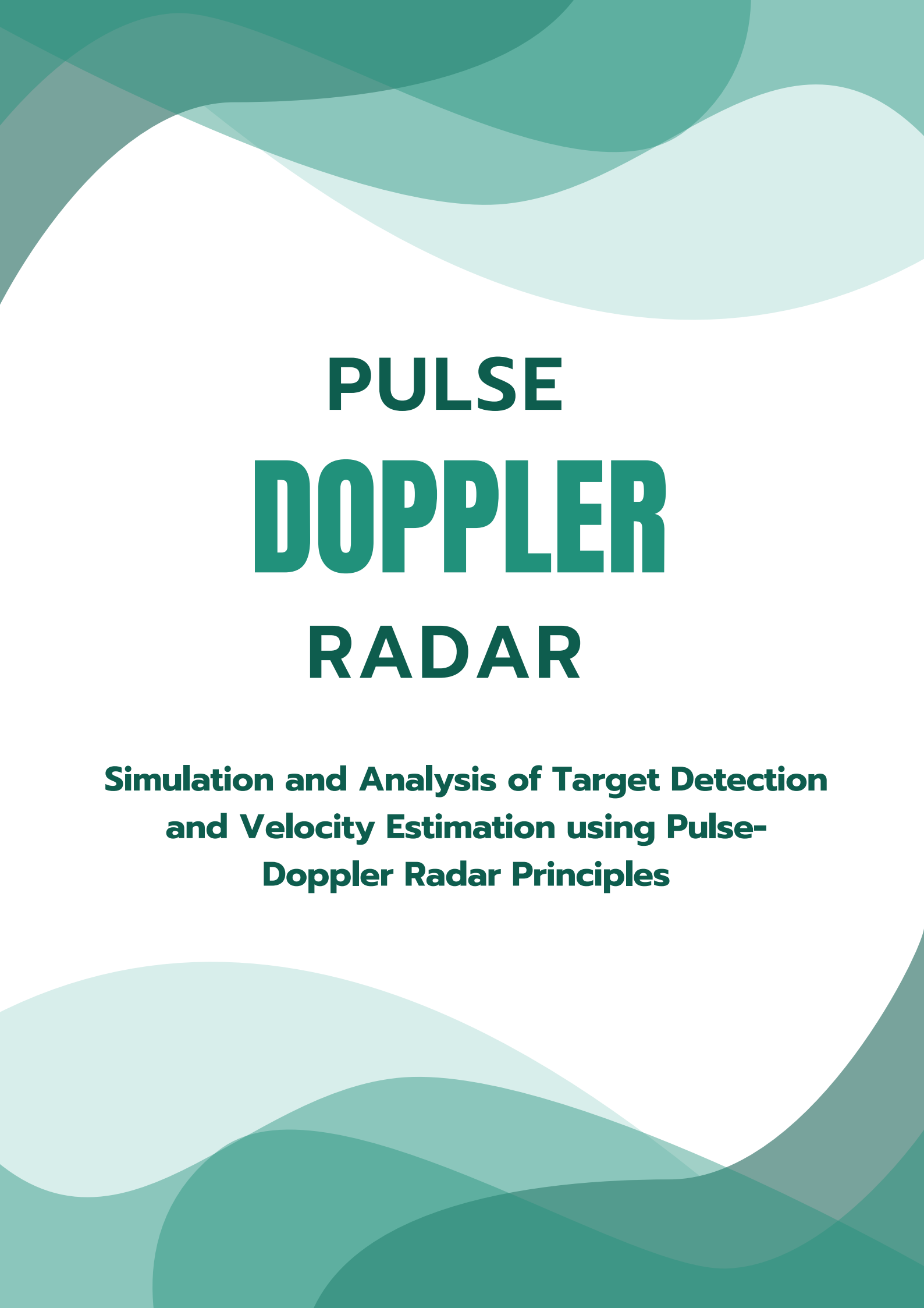




PULSE DOPPLER RADAR

**Simulation and Analysis of Target Detection
and Velocity Estimation using Pulse-
Doppler Radar Principles**

Acknowledgment

We would like to express our deepest gratitude to

Dr. Michael Melek & Dr. Mohammed Abdelghany

By challenging us to combine analytical depth with genuine human effort, they laid the intellectual groundwork for this process and motivated us throughout this project.

Our Beloved Team

Name	عبدالله طارق سيد احمد	أحمد عصام عفيفي
Code	91240462	91240101

Introduction

The Document has 2 Parts One With Positive SNR and Other With Negative SNR

Radar systems are widely used for detecting and tracking targets by transmitting electromagnetic waves and analyzing their echoes. In this report, a **pulse-Doppler radar system** operating in the **X-band (8–12 GHz)** is designed. The system parameters are selected, key performance metrics are calculated, and a target scenario with two moving targets is simulated. The transmitted and received signals are generated, including Doppler effects and noise, and the results are visualized using time-domain and range-time plots

2.Radar System Parameters

Parameter	Symbol	Value
Speed of light	c	$3 * 10^8$ m/s
Carrier frequency	f_c	76.5 GHz
Wavelength	$\lambda = c/f_c$	3.92 mm
Pulse Repetition Interval	PRI	4 μ s
Pulse Repetition Frequency	$1/PRI$	250 KHz
Pulse width	T_p	5ns
Number of pulses	(N)	256
Sample Frequency	Fs	2 GHz

3. Radar Performance Calculations

3.1 Maximum Unambiguous Range

The maximum unambiguous range is given by: $R_{\max} = \frac{c \times \text{PRI}}{2} = \frac{(3 \times 10^8) \times (\text{PRI})}{2} = \mathbf{600\text{ m}}$

3.2 Maximum Unambiguous Velocity

The maximum unambiguous velocity is calculated as: $v_{\max} = \frac{c \times \text{PRF}}{4 \times f_c} = \frac{(3 \times 10^8) \times \text{PRF}}{4 \times f_c} = \mathbf{245\text{ m/s}}$

3.3 Minimum Detectable Range (Blind Zone)

In real radar systems, the receiver cannot receive echoes while the transmitter is sending a pulse. This creates a blind zone near the radar called the minimum detectable range, denoted as $R_{\min}$

The minimum range is calculated using: $R_{\min} = \frac{c \times T_p}{2} = \frac{3 \times 10^8 \times 5 \times 10^{-9}}{2} = \mathbf{0.75\text{ m}}$

This means the radar cannot detect any target closer than 75 meters

4. Target Scenario

A scenario with **two moving targets** is designed. The selected target parameters are realistic:

Target	Range (m)	Velocity (m/s)
Target A	150	-12.56
Target B	45.5	50

5. Transmitted Signal Model

A **Rectangular** pulse is used as the transmitted waveform

6. Received Signal Model

The received signal is formed by summing echoes from both targets. Each echo includes:

- **Time delay** due to target range:

$$\tau = \frac{2R}{c}$$

- **Doppler frequency shift:**

$$f_d = \frac{2v}{\lambda}$$

- **Doppler phase accumulation across pulses:**

$$\phi[n] = 2\pi f_d \times n \times \text{PRI}$$

7. Noise Model

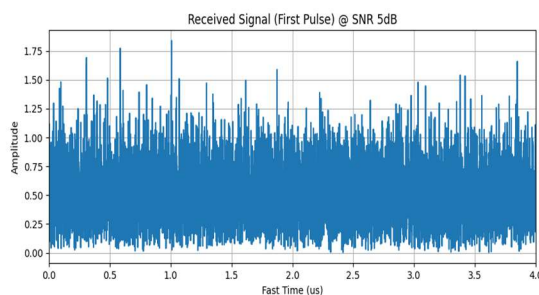
Additive White Gaussian Noise is added to the received signal with a selected **Signal-to-Noise Ratio (SNR)**:

$$\text{SNR} = 10$$

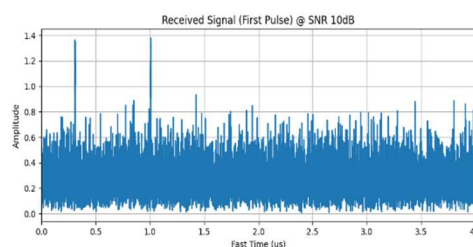
Why 10 dB is a Good Choice (Practical Explanation)

From a practical radar point of view:

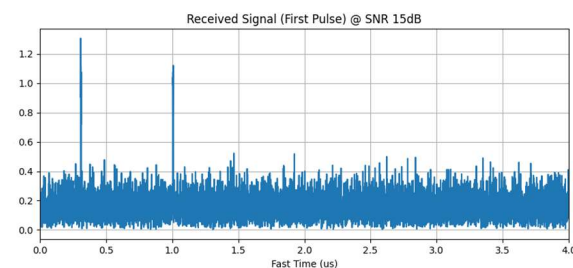
- **At low SNR (≈ 5 dB):**
Noise power is close to signal power, which can cause false targets or missed detections
- **At high SNR (≈ 15 dB):**
The received signal is very clear, and advanced processing techniques are not really needed
- **At SNR = 10 dB:**
The received signal looks noisy in the time domain, but signal processing techniques such as **matched filtering** can clearly extract the target. This makes it an ideal choice for demonstrating the effectiveness of the radar processing algorithm used in the code



SNR=5



SNR=10



SNR=15

8. Simulation Results

8.1 First Transmitted Pulse

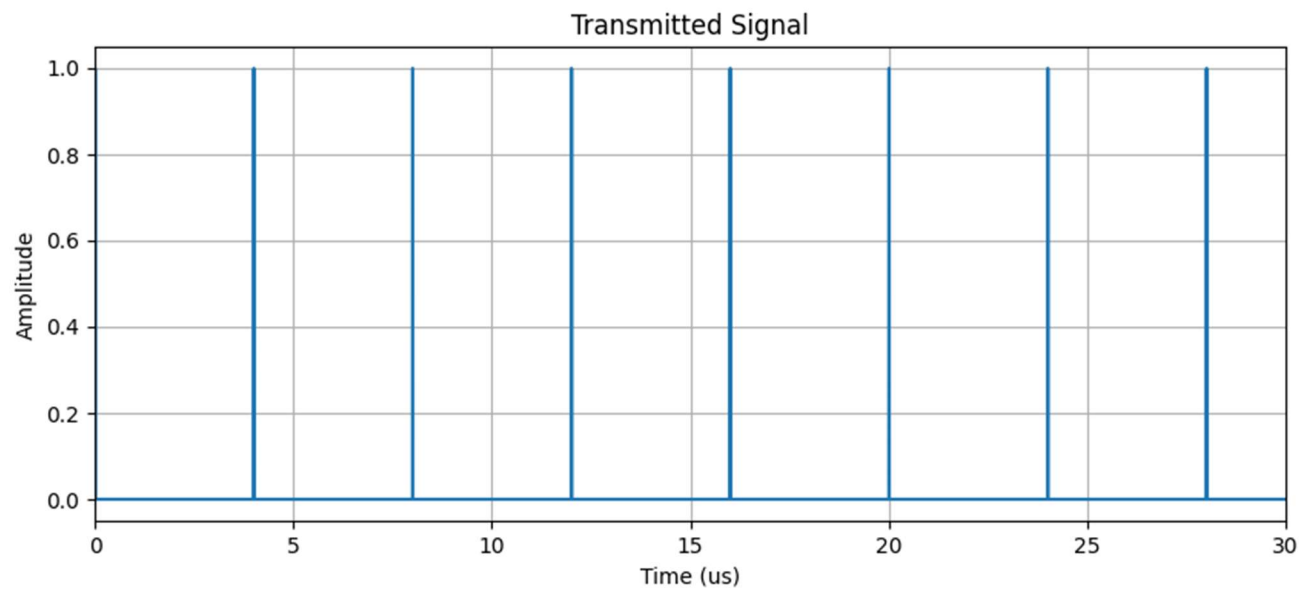


Figure 1: Transmitted pulse

8.2 Received Signal (Single Pulse)

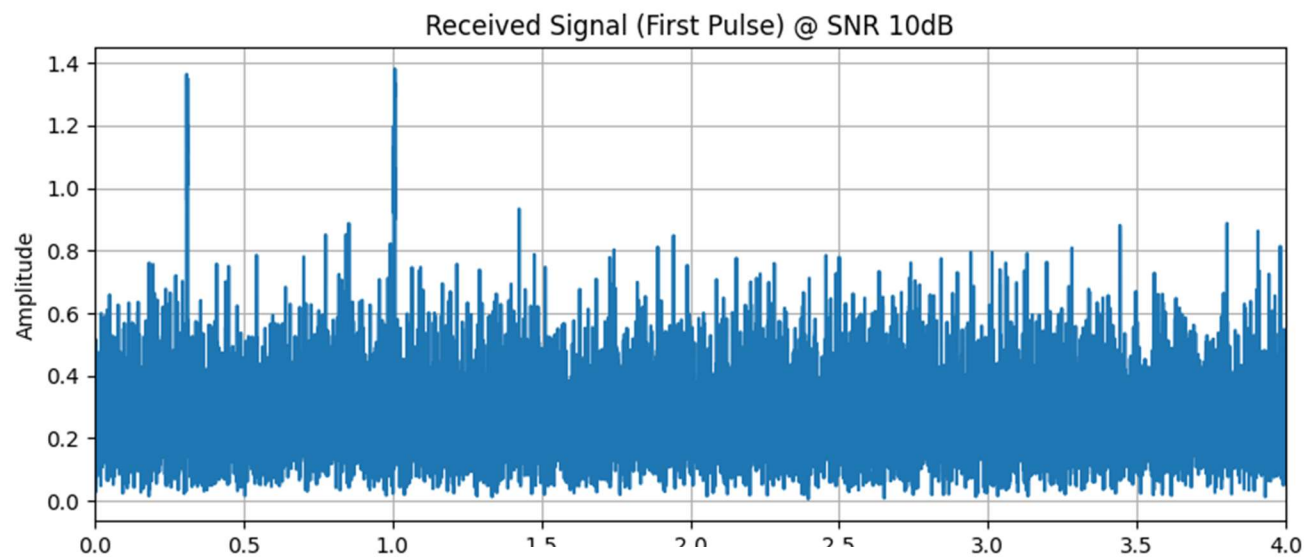
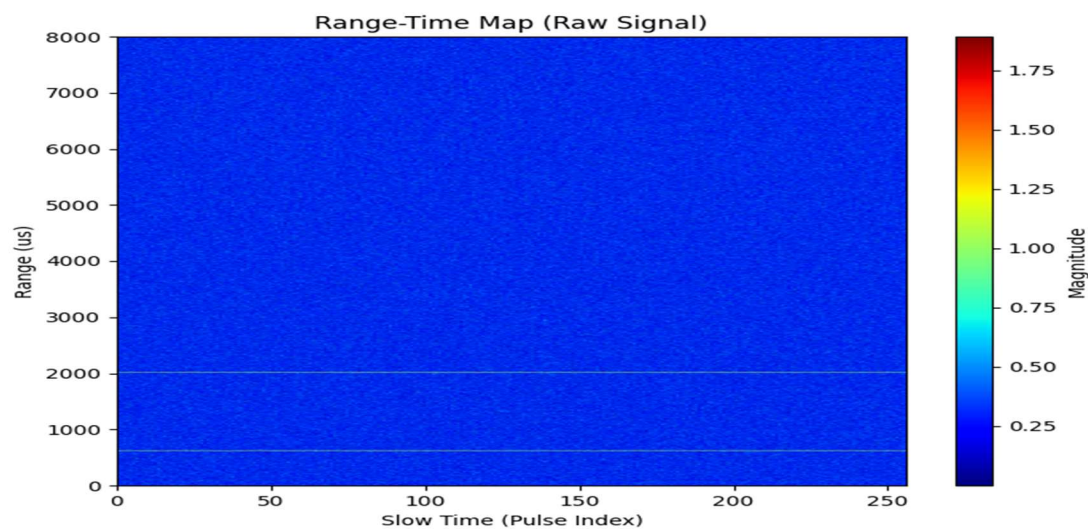
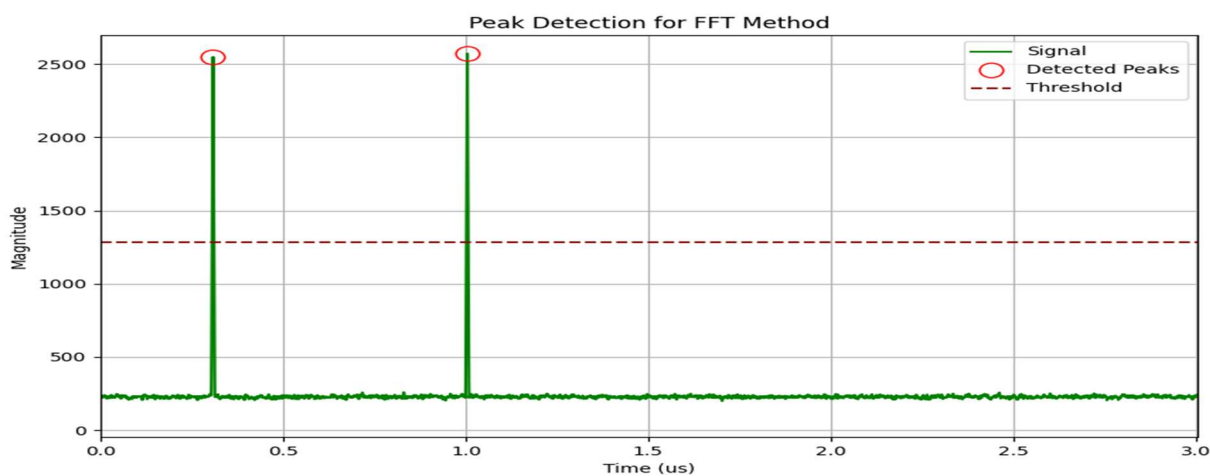
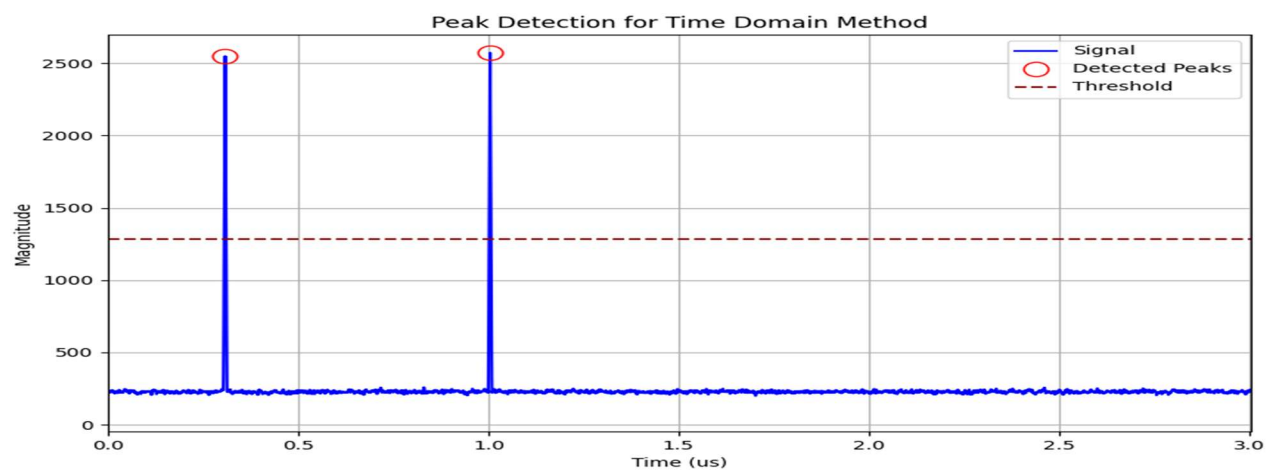
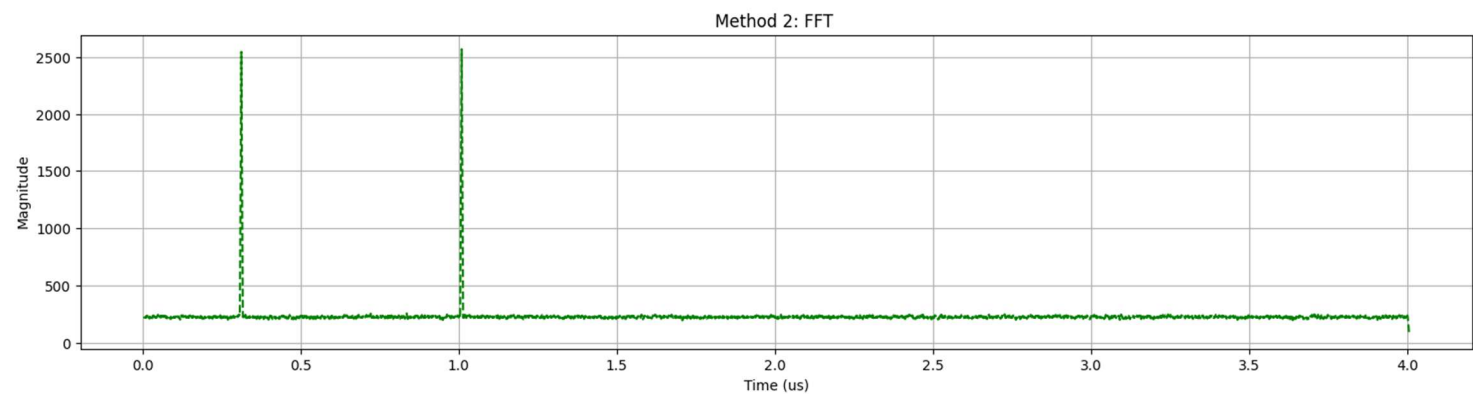
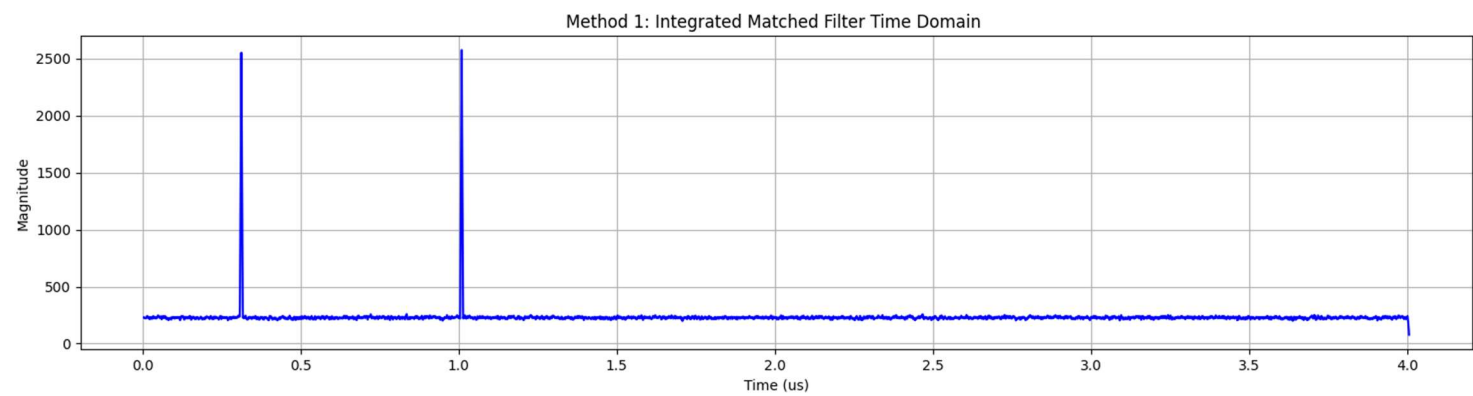


Figure 2: Received signal for one pulse

8.3 Range-Time Diagram



b. Range Detection: Method Comparison



3:Analysis Results

$$\text{Detected range} = R = \frac{c \times (T_d - \text{delay})}{2}$$

Method	Time Detected(μ s) (First Target)	Time Detected(μ s) (Second Target)	Start Delay(ns)
Matched Filter (Time Domain)	0.308	1.0045	5
FFT-Based Method	0.308	1.0045	

Target	True Range (m)	Detected Range – Matched Filter (m)	Detected Range – FFT Method (m)
Target 1	150	149.925	149.925
Target 2	45.5	45.45	45.45

Method	Processing Time (ms)
Matched Filter (Time Domain)	0.7739
FFT-Based Method	3.5892

4.Analysis Conclusion

Equivalence:

Both methods produce nearly identical detection results. The time-domain Matched Filter and FFT show the same peaks, confirming that convolution in the time domain is mathematically equivalent to multiplication in the frequency domain. The resulting plots are essentially identical.

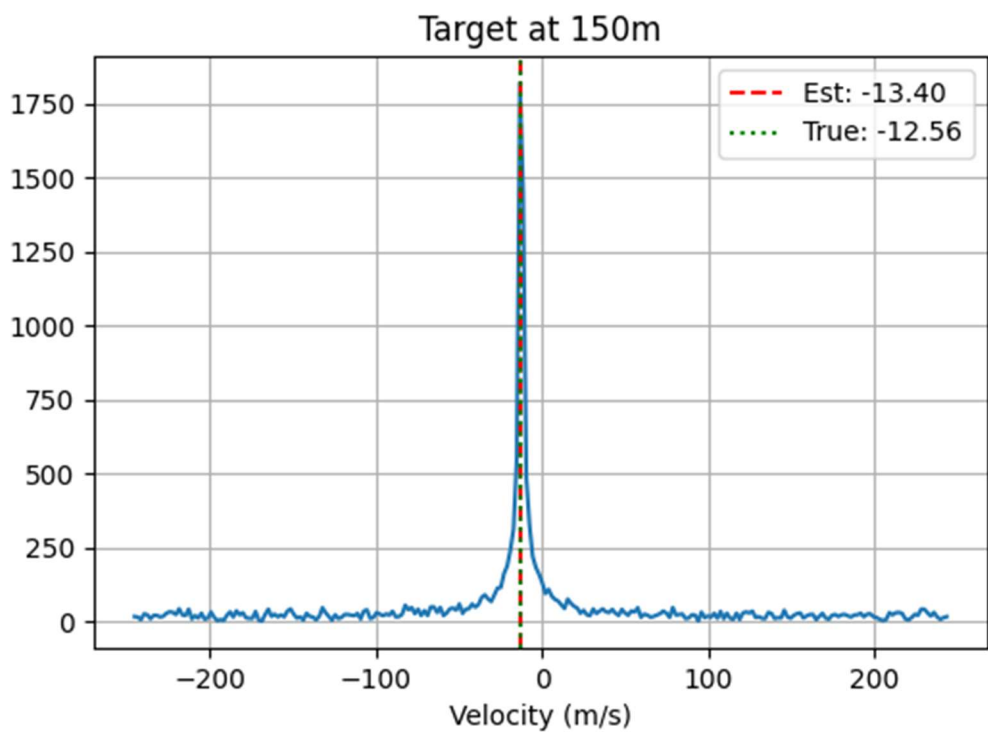
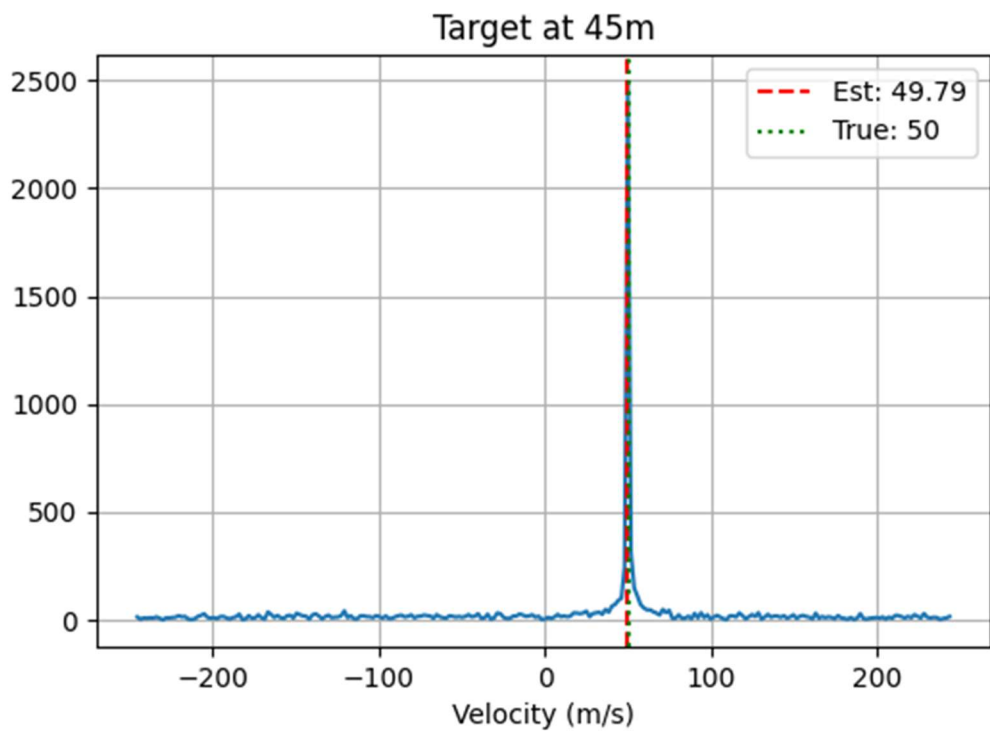
Speed:

The FFT is generally faster, especially as the dataset size or frequency increases. For small datasets or low-frequency signals, the time-domain Matched Filter may actually be faster due to lower overhead. However, as the signal frequency or dataset size grows, FFT processing becomes significantly faster thanks to its $O(N \log N)$ complexity, compared to the $O(N^2)$ complexity of direct time-domain convolution.

Practical Recommendation:

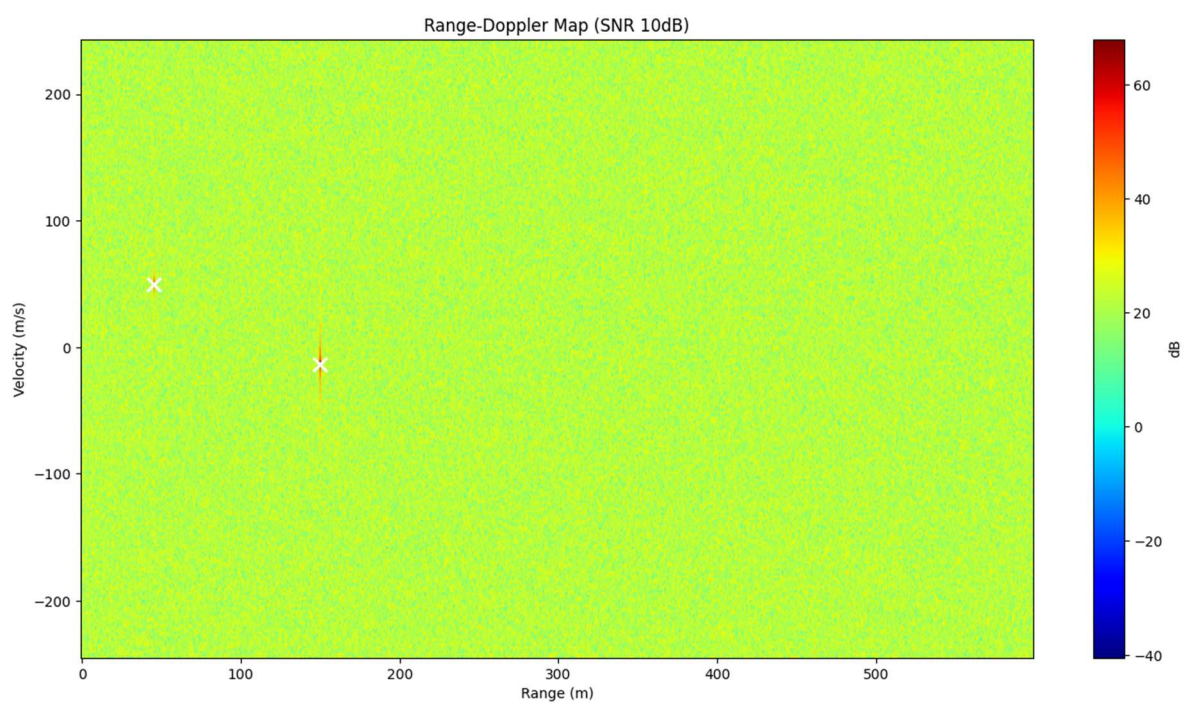
- **Time-domain Matched Filter:** simple and intuitive; works well for short pulses, low-frequency signals, or small datasets.
- **FFT-based method:** preferred for long signals, high-frequency pulses, multiple targets, or real-time applications due to faster computation at higher frequencies and larger datasets.

c. Velocity Estimation Using Doppler Analysis



Target	True Velocity (m/s)	Estimated Velocity(m/s)	Error	Direction
Target 1	-12.56	-13.4	0.84	Receding
Target 2	50	49.79	0.21	approaching

d. Range-Doppler Map Generation



Second part of the document with negative SNR

2. Radar System Parameters

Parameter	Symbol	Value
Speed of light	c	3×10^8 m/s
Carrier frequency	f_c	76.5 GHz
Wavelength	$\lambda = c/f_c$	3.92 mm
Pulse Repetition Interval	PRI	5 μ s
Pulse Repetition Frequency	$1/PRI$	200 KHz
Pulse width	T_p	4ns
Number of pulses	(N)	1024
Sample Frequency	Fs	4 GHz

3. Radar Performance Calculations

3.1 Maximum Unambiguous Range

The maximum unambiguous range is given by: $R_{\max} = \frac{c \times \text{PRI}}{2} = \frac{(3 \times 10^8) \times (\text{PRI})}{2} = \mathbf{750 \text{ m}}$

3.2 Maximum Unambiguous Velocity

The maximum unambiguous velocity is calculated as: $v_{\max} = \frac{c \times \text{PRF}}{4 \times f_c} = \frac{(3 \times 10^8) \times \text{PRF}}{4 \times f_c} = \mathbf{196 \text{ m/s}}$

3.3 Minimum Detectable Range (Blind Zone)

In real radar systems, the receiver cannot receive echoes while the transmitter is sending a pulse. This creates a blind zone near the radar called the minimum detectable range, denoted as $R_{\min}$

The minimum range is calculated using: $R_{\min} = \frac{c \times T_p}{2} = \frac{3 \times 10^8 \times 4 \times 10^{-9}}{2} = 0.6 \text{ m}$

This means the radar cannot detect any target closer than 0.75 meters (as stated in the description)

4. Target Scenario

A scenario with **two moving targets** is designed. The selected target parameters are realistic:

Target	Range (m)	Velocity (m/s)
--------	-----------	----------------

Target A	45.5	-30
Target B	150	65

5. Transmitted Signal Model

A **Rectangular** pulse is used as the transmitted waveform

6. Received Signal Model

The received signal is formed by summing echoes from both targets. Each echo includes:

- **Time delay** due to target range:

$$\tau = \frac{2R}{c}$$

- **Doppler frequency shift:**

$$f_d = \frac{2v}{\lambda}$$

- **Doppler phase accumulation across pulses:**

$$\phi[n] = 2\pi f_d \times n \times \text{PRI}$$

7. Noise Model

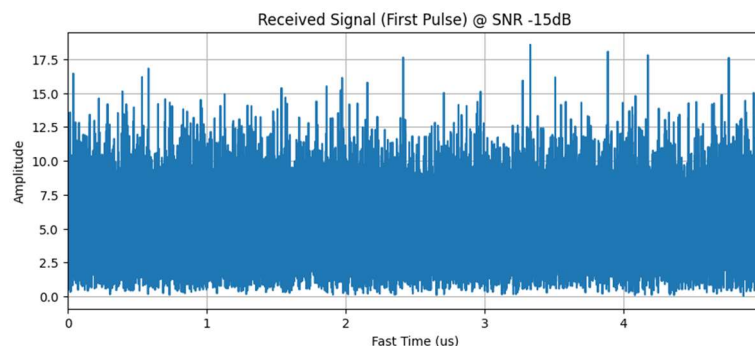
Additive White Gaussian Noise is added to the received signal with a selected **Signal-to-Noise Ratio (SNR)**:

$$\text{SNR} = -15$$

At Extremely Low SNR (≈ -20 dB): The noise power is 100 times stronger than the signal power. At this level, the noise floor is so high that even with integration, the targets often remain buried or result in unstable detections (false alarms), making it impractical for this specific simulation setup.

At Higher SNR (≈ -5 dB): While still negative, the signal is relatively easy to recover. The noise is not strong enough to truly challenge the system, meaning advanced processing techniques (like Pulse Compression) aren't strictly necessary to see the target.

At SNR = -15 dB: The noise power is significantly higher than the signal power (approx. 30x stronger). In the time domain, the received signal appears completely as random noise with no visible targets. However, **Matched Filtering and Coherent Integration** provide enough "Processing Gain" to pull the target out of the noise floor. This makes it the ideal choice to demonstrate the power and effectiveness of the radar processing algorithms used in the code.



SNR=-15

8.1 First Transmitted Pulse

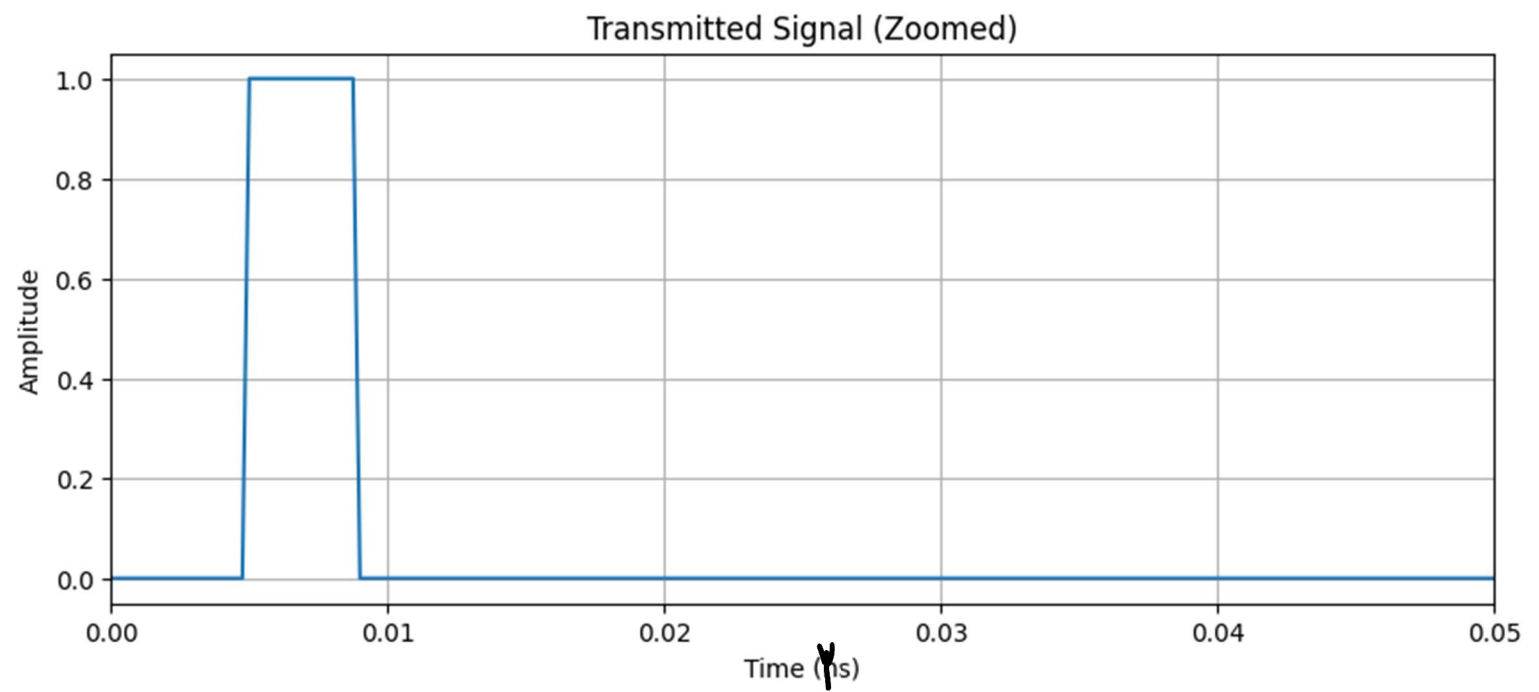


Figure 1: First Transmitted pulse

8.2 Received Signal (Single Pulse)

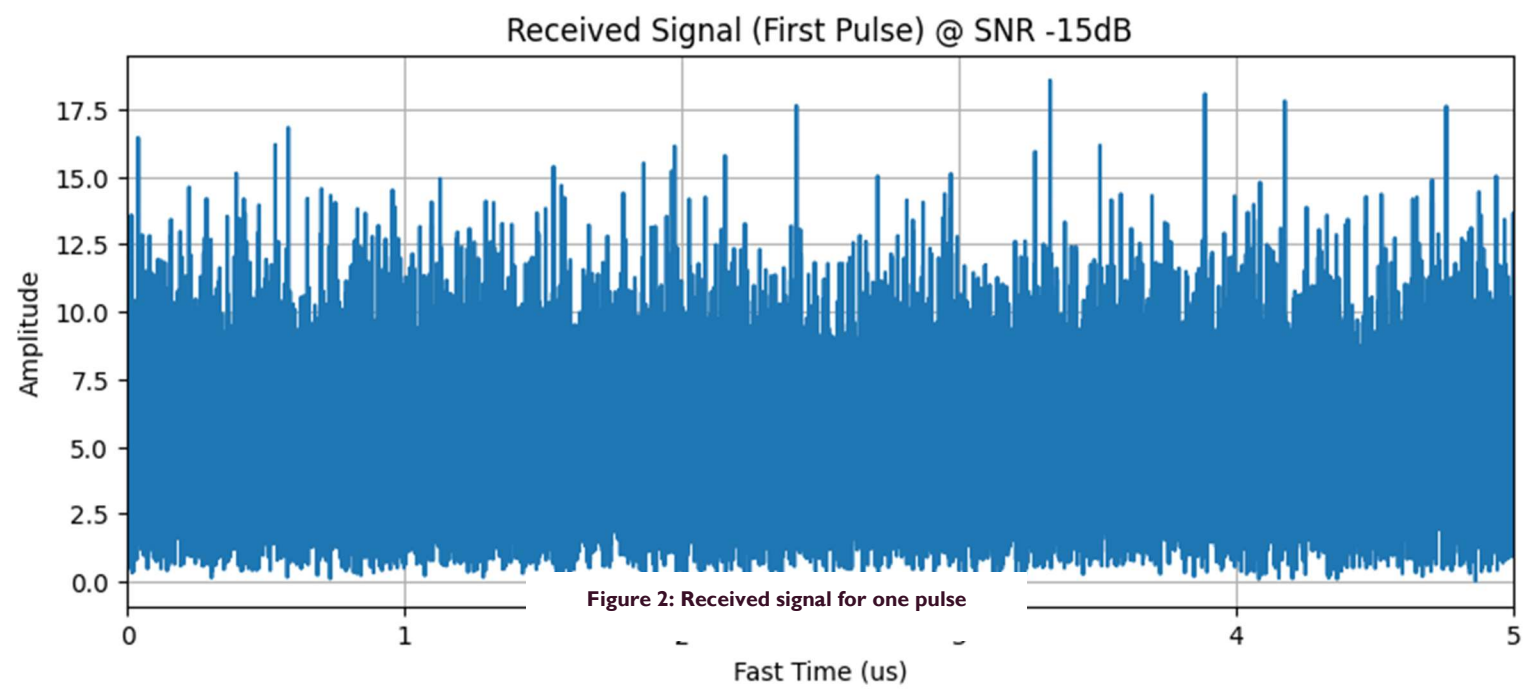


Figure 2: Received signal for one pulse

8.3 Range-Time Diagram

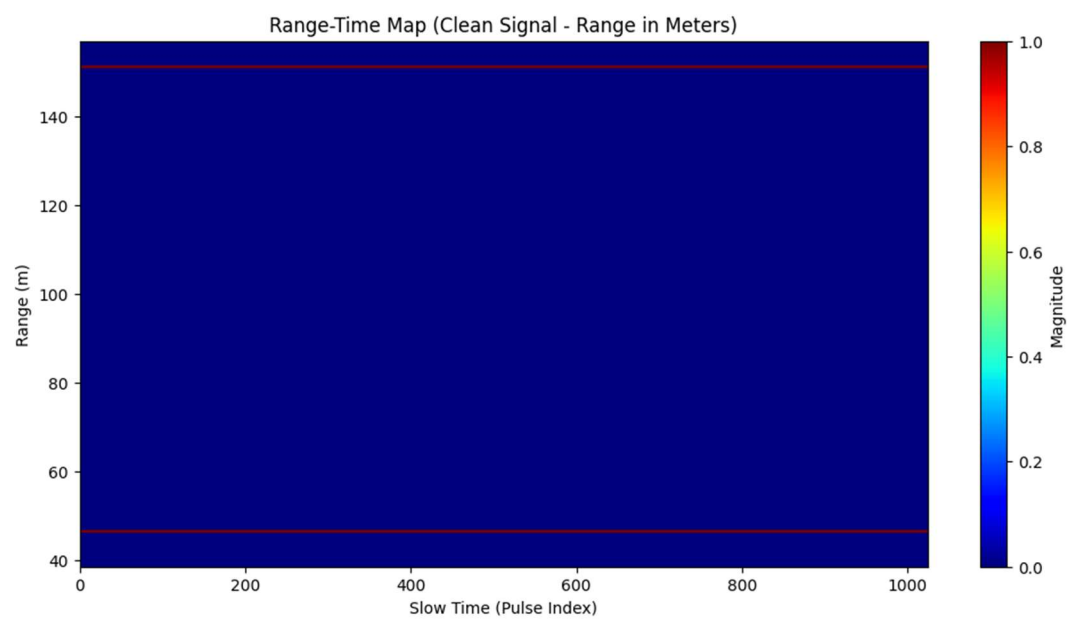
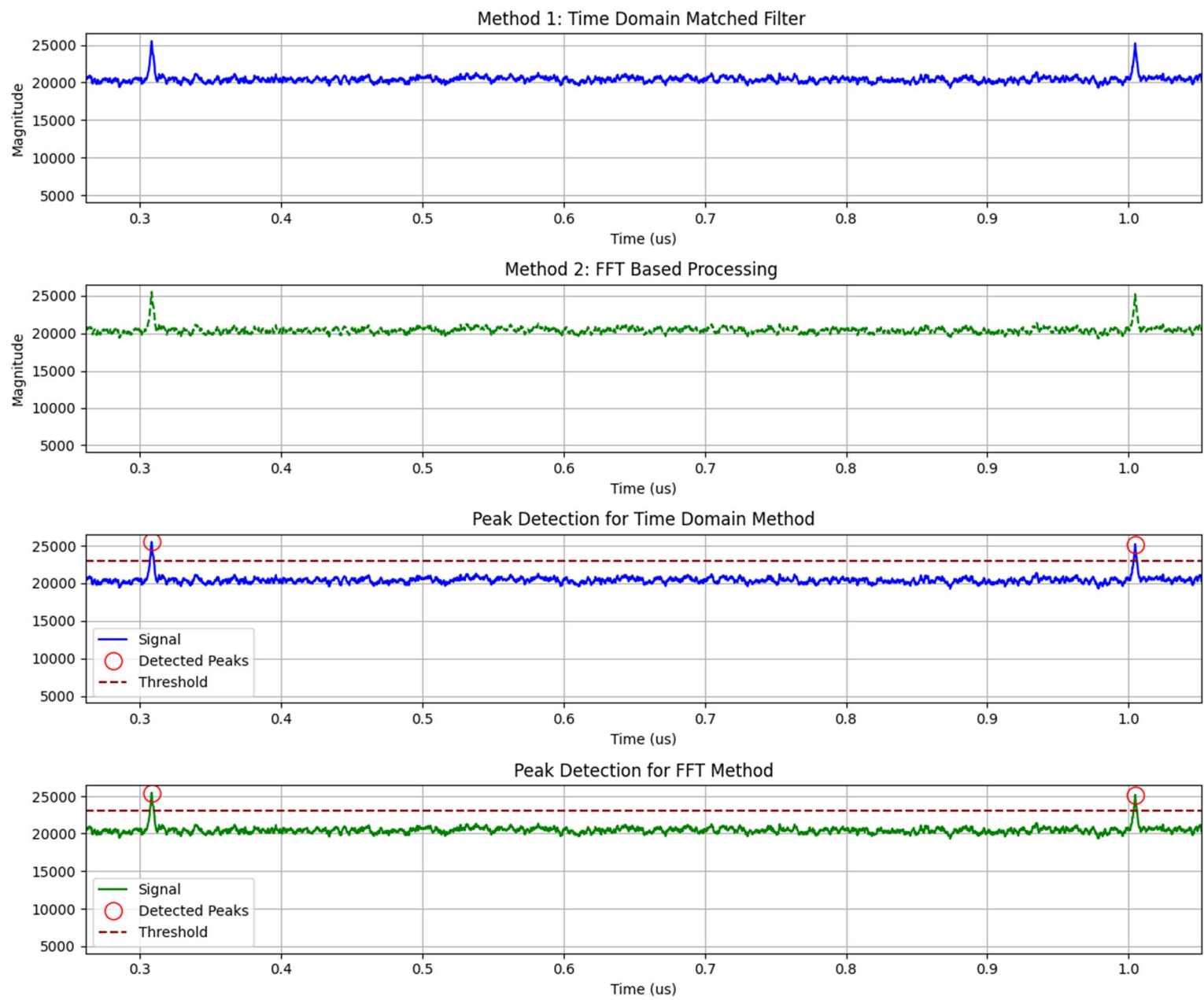
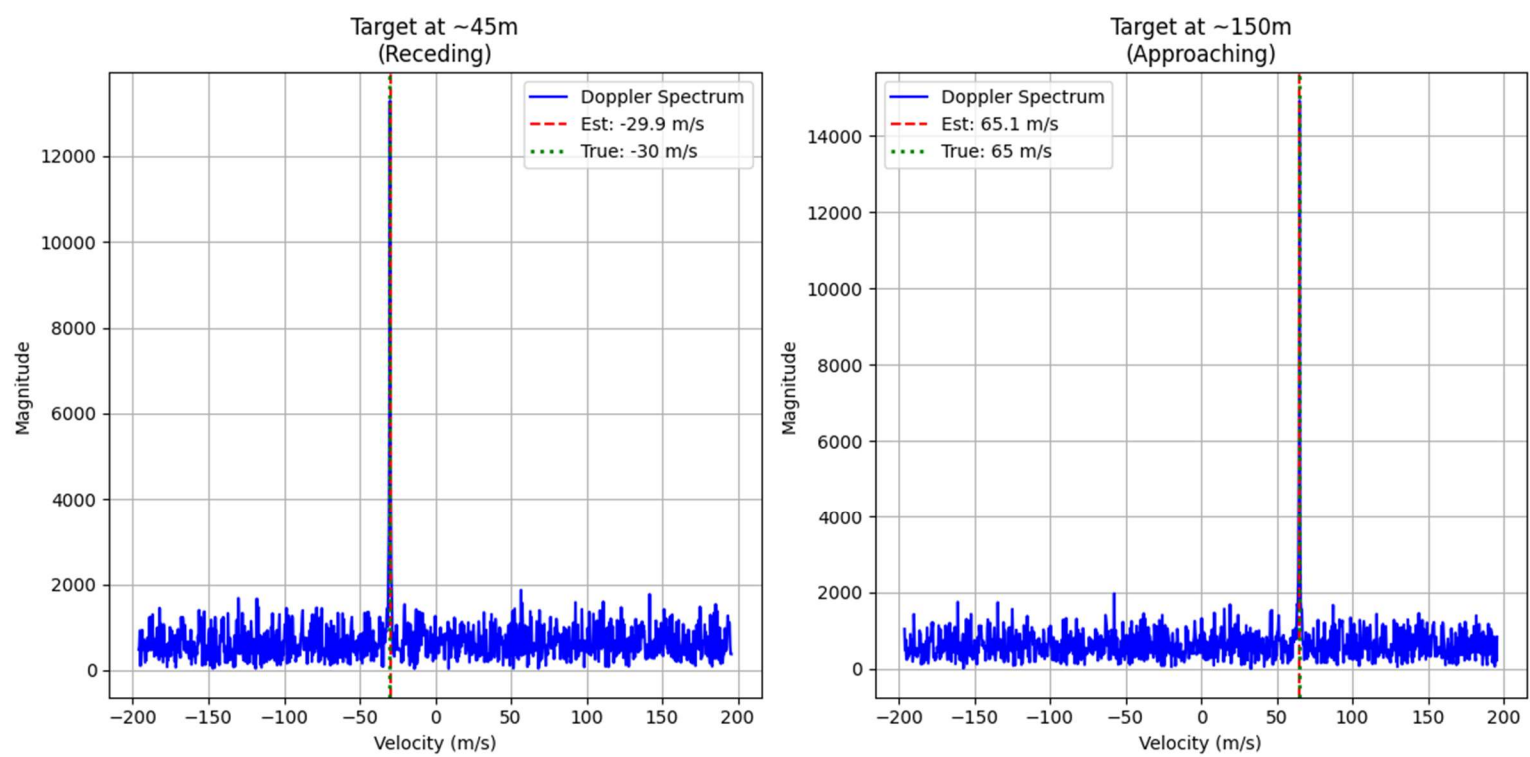


Figure 3: Range-Time diagram of received echoes

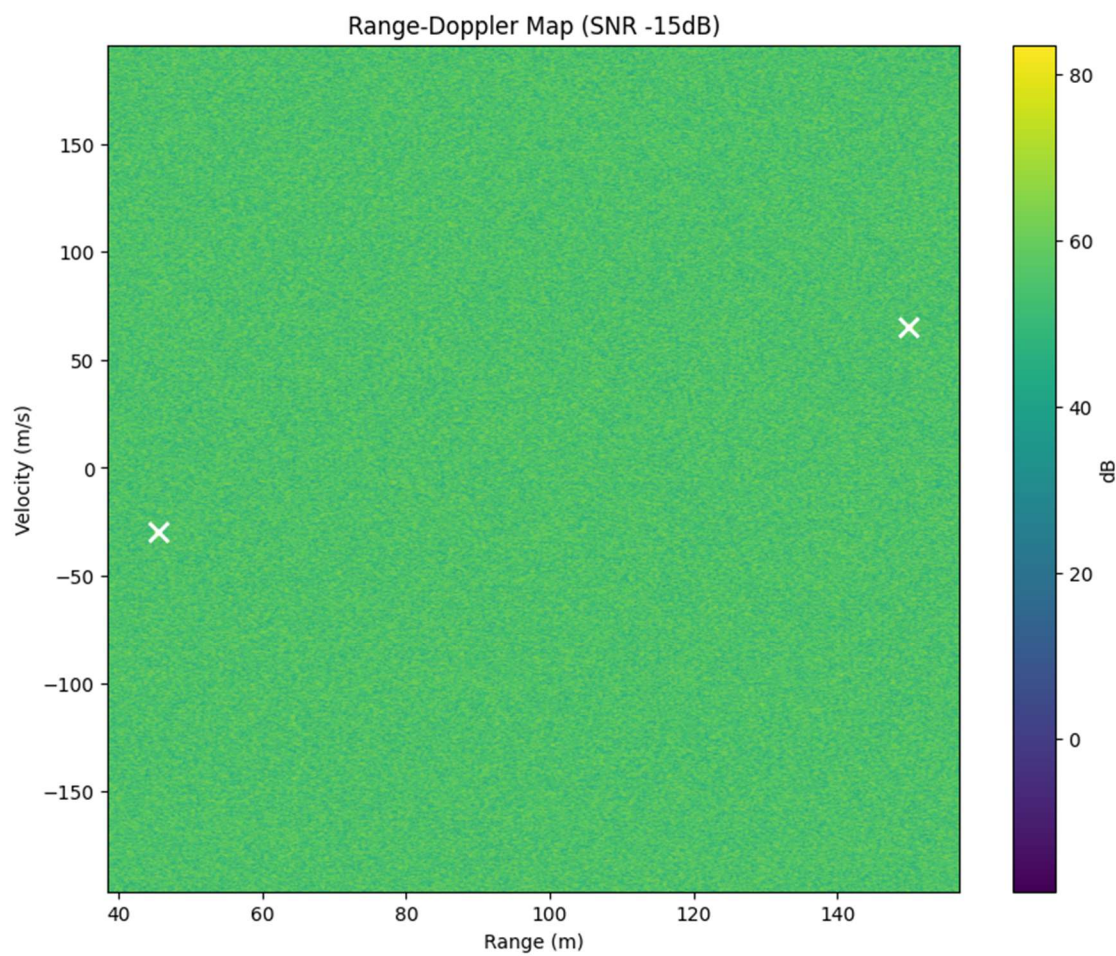
b. Range Detection: Method Comparison



c. Velocity Estimation Using Doppler Analysis



d. Range-Doppler Map Generation



References:

Python Libraries:

<https://matplotlib.org/stable/>

<https://numpy.org/>

YouTube Videos:

<https://youtu.be/NtyU6aKZ-cY?si=Gsaex5SeVWtqE9F>

https://youtu.be/UO98lJQ3QGI?si=Ol_GqJy88m4Tae-G

<https://www.youtube.com/watch?v=OnDIGlvTGcc>

<https://youtu.be/VXU4LSAQDSc?si=uRe0xMSSBzBhK6rp>

Google Search:

<https://radarsimx.com/2019/05/16/doppler-radar/>

<https://medium.com/%40itberrios6/introduction-to-radar-part-2-8a332066917e>

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

<https://www.mathworks.com/help/phased/ug/range-doppler-response.html>

```

import numpy as np

import matplotlib.pyplot as plt

import time

from scipy.signal import find_peaks


c = 3e8

Fc = 76.5e9

lamda = c / Fc

PRI = 4e-6

Tp = 5e-9

Fs = 2e9

startDelayTime = 5e-9

pulsesNum = 256

SNR = 10

Ts = 1 / Fs

margin = 2


samplesPerPulse = int(PRI * Fs)

fastTimeArray = np.arange(0, samplesPerPulse) * Ts


targetsDistance = [150, 45.5]

targetsVelocity = [-12.56, 50]


timeArray = np.arange(0, pulsesNum * PRI + startDelayTime, Ts)

transmittedSignal = np.zeros_like(timeArray)


for pulseIndex in range(pulsesNum):

    startTime = startDelayTime + pulseIndex * PRI

    transmittedSignal[(timeArray >= startTime) & (timeArray < startTime + Tp)] = 1


receivedSignalArray = np.zeros((pulsesNum, samplesPerPulse), dtype=complex)


for pulseIndex in range(pulsesNum):

```

```

currentPulseSignal = np.zeros(samplesPerPulse, dtype=complex)

for i in range(len(targetsDistance)):

    r = targetsDistance[i]

    v = targetsVelocity[i]

    fd = (2 * v) / lamda

    tau = (2 * r / c) + startDelayTime

    delaySamples = int(tau * Fs)

    pulseWidthSamples = int(Tp * Fs)

    if delaySamples + pulseWidthSamples < samplesPerPulse:

        phase = np.exp(1j * 2 * np.pi * fd * (pulseIndex * PRI + startDelayTime))

        currentPulseSignal[delaySamples: delaySamples + pulseWidthSamples] += 1 * phase

receivedSignalArray[pulseIndex, :] = currentPulseSignal

noisePower = 10 ** (-SNR / 10)

noise = np.sqrt(noisePower / 2) * (

    np.random.randn(pulsesNum, samplesPerPulse) +

    1j * np.random.randn(pulsesNum, samplesPerPulse)

)

receivedSignalWithNoise = receivedSignalArray + noise

plt.figure(figsize=(10, 4))

plt.plot(timeArray * 1e6, transmittedSignal)

plt.xlabel("Time (us)")

plt.ylabel("Amplitude")

plt.xlim(0, 30)

plt.grid()

plt.title("Transmitted Signal")

```

```

plt.figure(figsize=(10, 4))

plt.plot((fastTimeArray * 1e6), np.abs(receivedSignalWithNoise[0]))

plt.xlabel("Fast Time (us)")

plt.ylabel("Amplitude")

plt.xlim(0, PRI * 1e6)

plt.grid()

plt.title(f"Received Signal (First Pulse) @ SNR {SNR}dB")


plt.figure(figsize=(8, 6))

plt.imshow(

    np.abs(receivedSignalWithNoise).T,

    aspect="auto",

    extent=[0, pulsesNum, 0, PRI / Ts],

    cmap="jet",

    origin='lower'

)

plt.xlabel("Slow Time (Pulse Index)")

plt.ylabel("Range (us)")

plt.colorbar(label="Magnitude")

plt.title("Range-Time Map (Raw Signal)")

plt.show()


pulseTemplate = np.ones(int(Tp * Fs))

pulseTemplateFlipped = np.flip(pulseTemplate)


rangeCompressed = np.zeros_like(receivedSignalWithNoise)


for n in range(pulsesNum):

    convResult = np.convolve(receivedSignalWithNoise[n], pulseTemplateFlipped, mode='full')

    rangeCompressed[n, :] = convResult[len(pulseTemplate) - 1: len(pulseTemplate) - 1 + samplesPerPulse]


t0 = time.time()

integratedSignalTime = np.sum(np.abs(rangeCompressed), axis=0)

```

```

t1 = time.time()

timeMF = (t1 - t0) * 1000

plt.figure(figsize=(8, 6))

plt.imshow(

    np.abs(rangeCompressed).T,

    aspect="auto",

    extent=[0, pulsesNum, 0, PRI * 1e6],

    cmap="jet",

    origin='lower'

)

plt.xlabel("Slow Time (Pulse Index)")

plt.ylabel("Fast Time / Range (us)")

plt.colorbar(label="Magnitude")

plt.title(f"Range-Time Map (Matched Filter Output)")

plt.show()


startTimeFFT = time.time()

integratedSignalFFT = np.zeros(samplesPerPulse)

nFFT = 2 ** (int(np.log2(samplesPerPulse)) + 1)

pulseTemplateFFT = np.fft.fft(pulseTemplate, nFFT)


for pulseIdx in range(pulsesNum):

    receivedFFT = np.fft.fft(receivedSignalWithNoise[pulseIdx, :], nFFT)

    matchedFilterFFT = receivedFFT * np.conj(pulseTemplateFFT)

    matchedFilterOutput = np.fft.ifft(matchedFilterFFT)

    integratedSignalFFT += np.abs(matchedFilterOutput[:samplesPerPulse])


endTimeFFT = time.time()

processingTimeFFT = (endTimeFFT - startTimeFFT) * 1000


peakThreshold = 0.5 * np.max(integratedSignalTime)

peaksIdxMF, _ = find_peaks(integratedSignalTime, height=peakThreshold, distance=int(Tp * Fs))

```

```

peakTimesMF = fastTimeArray[peaksIdxMF]

detectedRangesMF = (peakTimesMF - startDelayTime) * c / 2


peaksIdxFFT, _ = find_peaks(integratedSignalFFT, height=peakThreshold, distance=int(Tp * Fs))

peakTimesFFT = fastTimeArray[peaksIdxFFT]

detectedRangesFFT = (peakTimesFFT - startDelayTime) * c / 2


plt.figure(figsize=(10, 8))


plt.subplot(2, 1, 1)

plt.plot((fastTimeArray * 1e6) + (startDelayTime * 1e6), integratedSignalTime, 'b', label='Integrated')

plt.title(f"Method 1: Integrated Matched Filter Time Domain ")

plt.xlabel("Time (us)")

plt.ylabel("Magnitude")

plt.grid()


plt.subplot(2, 1, 2)

plt.plot((fastTimeArray * 1e6) + (startDelayTime * 1e6), integratedSignalFFT, 'g--', label='Integrated')

plt.title(f"Method 2: FFT ")

plt.xlabel("Time (us)")

plt.ylabel("Magnitude")

plt.grid()


plt.tight_layout()

plt.show()


if len(peaksIdxMF) > 0:

    minTimeDomain = peakTimesMF.min() * 1e6

    maxTimeDomain = peakTimesMF.max() * 1e6


plt.figure(figsize=(10, 6))

plt.plot(fastTimeArray * 1e6, integratedSignalTime, color='blue', label='Signal')

plt.plot(peakTimesMF * 1e6, integratedSignalTime[peaksIdxMF], marker='o', linestyle='none', fillstyle='none',

```

```

        color='red', markersize=12, label='Detected Peaks')

plt.axhline(peakThreshold, color='darkred', linestyle='--', label='Threshold')

plt.title('Peak Detection for Time Domain Method')

plt.xlabel('Time (us)')

plt.ylabel('Magnitude')

plt.legend()

plt.grid(True)

plt.xlim(0 , maxTimeDomain + margain)

plt.show()

if len(peaksIdxFFT) > 0:

    minFft = peakTimesFFT.min() * 1e6

    maxFft = peakTimesFFT.max() * 1e6

    plt.figure(figsize=(10, 6))

    plt.plot(fastTimeArray * 1e6, integratedSignalFFT, color='green', label='Signal')

    plt.plot(peakTimesFFT * 1e6, integratedSignalFFT[peaksIdxFFT], color='red', marker='o', fillstyle='none',

             linestyle='none', markersize=12, label='Detected Peaks')

    plt.axhline(peakThreshold, color='darkred', linestyle='--', label='Threshold')

    plt.title('Peak Detection for FFT Method')

    plt.xlabel('Time (us)')

    plt.ylabel('Magnitude')

    plt.legend()

    plt.grid(True)

    plt.xlim(0, maxFft + margain)

    plt.show()

N_Doppler = pulsesNum

dopplerFreq = np.fft.fftshift(np.fft.fftfreq(N_Doppler, d=PRI))

velocityAxis = (dopplerFreq * c) / (2 * Fc)

detectedVelocities = []

```

```

for i, rangeIdx in enumerate(peaksIdxMF):

    slowTimeSignal = rangeCompressed[:, rangeIdx]

    dopplerSpectrum = np.fft.fftshift(np.fft.fft(slowTimeSignal, n=N_Doppler))

    magnitudeSpectrum = np.abs(dopplerSpectrum)

    peakDopplerIdx = np.argmax(magnitudeSpectrum)

    estVelocity = velocityAxis[peakDopplerIdx]

    detectedVelocities.append(estVelocity)

    detectedR = detectedRangesMF[i]

    closestTrueIdx = np.argmin(np.abs(np.array(targetsDistance) - detectedR))

    trueV = targetsVelocity[closestTrueIdx]

    plt.figure(figsize=(6, 4))

    plt.plot(velocityAxis, magnitudeSpectrum)

    plt.axvline(estVelocity, color='r', linestyle='--', label=f'Est: {estVelocity:.2f}')

    plt.axvline(trueV, color='g', linestyle=':', label=f'True: {trueV}')

    plt.title(f"Target at {detectedR:.0f}m")

    plt.xlabel("Velocity (m/s)")

    plt.legend()

    plt.grid()

    plt.show()

rangeDopplerMap = np.fft.fft(rangeCompressed, n=N_Doppler, axis=0)

rangeDopplerMapShifted = np.fft.fftshift(rangeDopplerMap, axes=0)

rangeDopplerMapDb = 20 * np.log10(np.abs(rangeDopplerMapShifted) + 1e-10)

rangeAxis = (fastTimeArray - startDelayTime) * c / 2

plt.figure(figsize=(10, 8))

plt.imshow(rangeDopplerMapDb, aspect='auto', extent=[rangeAxis[0], rangeAxis[-1], velocityAxis[0], velocityAxis[-1]],

            origin='lower', cmap='jet')

```



```

plt.colorbar(label='dB')

if len(detectedRangesMF) > 0:

    plt.plot(detectedRangesMF, detectedVelocities, 'wx', markersize=10, markeredgewidth=2)

plt.xlabel("Range (m)")

plt.ylabel("Velocity (m/s)")

plt.title(f"Range-Doppler Map (SNR {SNR}dB)")

plt.show()


print(f"Detected Peak Times (Method 1): {peakTimesMF}")

print(f"Detected Peak Times (Method 2): {peakTimesFFT}")


print(f"Detected Ranges (Method 1): {detectedRangesMF}")

print(f"Detected Ranges (Method 2): {detectedRangesFFT}")


print(f"\nProcessing Time (Method 1 - Time Domain): {timeMF:.3f} ms")

print(f"Processing Time (Method 2 - FFT Domain): {processingTimeFFT:.3f} ms")


print("\n--- VELOCITY COMPARISON ---")

print(f"{'Target':<10}{'True Velocity (m/s)':<25}{'Estimated Velocity (m/s)':<30}{'Error (m/s)':<20}{'Direction':<15}")

for i in range(len(detectedVelocities)):

    rightVelocity = detectedVelocities[i] if i < len(detectedVelocities) else None

    estVelocity = detectedVelocities[i]

    velocityError = estVelocity - rightVelocity

    direction = "Approaching" if estVelocity > 0 else "Receding"

    print(f"{i + 1:<10}{rightVelocity:<25.2f}{estVelocity:<30.2f}{velocityError:<20.2f}{direction:<15}")

```

Second Code of SNR with negative values like -15 db

```
import numpy as np

import matplotlib.pyplot as plt

import time

from scipy.signal import find_peaks


c = 3e8

Fc = 76.5e9

lamda = c / Fc

PRI = 5e-6

Tp = 4e-9

Fs = 4e9

startDelayTime = 5e-9

pulsesNum = 1024

SNR = -15

Ts = 1 / Fs

margin = 7


targetsDistance = [45.5, 150.0]

targetsVelocity = [-30, 65]


samplesPerPulse = int(PRI * Fs)

fastTimeArray = np.arange(0, samplesPerPulse) * Ts

timeArray = np.arange(0, pulsesNum * PRI + startDelayTime, Ts)


transmittedSignal = np.zeros_like(timeArray)


for pulseIndex in range(pulsesNum):

    startTime = startDelayTime + pulseIndex * PRI

    startIdx = int(startTime * Fs)

    endIdx = int((startTime + Tp) * Fs)
```

```

    if endIdx < len(transmittedSignal):

        transmittedSignal[startIdx:endIdx] = 1

receivedSignalArray = np.zeros((pulsesNum, samplesPerPulse), dtype=complex)

for pulseIndex in range(pulsesNum):

    currentPulseSignal = np.zeros(samplesPerPulse, dtype=complex)

    for i in range(len(targetsDistance)):

        r = targetsDistance[i]

        v = targetsVelocity[i]

        fd = (2 * v) / lamda

        tau = (2 * r / c) + startDelayTime

        delaySamples = int(tau * Fs)

        pulseWidthSamples = int(Tp * Fs)

        if delaySamples + pulseWidthSamples < samplesPerPulse:

            phase = np.exp(1j * 2 * np.pi * fd * (pulseIndex * PRI + startDelayTime))

            currentPulseSignal[delaySamples:delaySamples + pulseWidthSamples] += 1 * phase

    receivedSignalArray[pulseIndex, :] = currentPulseSignal

noisePower = 10 ** (-SNR / 10)

noiseArray = np.sqrt(noisePower / 2) * (

    np.random.randn(pulsesNum, samplesPerPulse) +

    1j * np.random.randn(pulsesNum, samplesPerPulse)

)

receivedSignalWithNoise = receivedSignalArray + noiseArray

plt.figure(figsize=(10, 4))

plt.plot(timeArray * 1e6, transmittedSignal)

plt.xlabel("Time (us)")

```

```

plt.ylabel("Amplitude")

plt.xlim(0, 0.05)

plt.grid()

plt.title("Transmitted Signal (Zoomed)")


min_dist_view = max(0, min(targetsDistance) - margain)

max_dist_view = max(targetsDistance) + margain

minTimeView = ((2 * min_dist_view / c) + startDelayTime) * 1e6

maxTimeView = ((2 * max_dist_view / c) + startDelayTime) * 1e6


plt.figure(figsize=(10, 4))

plt.plot(fastTimeArray * 1e6, np.abs(receivedSignalWithNoise[0]))

plt.xlabel("Fast Time (us)")

plt.ylabel("Amplitude")

plt.xlim(0, 4)

plt.grid()

plt.title(f"Received Signal (First Pulse) @ SNR {SNR}dB")


plt.figure(figsize=(12, 6))


max_range_m = (PRI * c) / 2


plt.imshow(

    np.abs(receivedSignalArray).T,

    aspect="auto",

    extent=[0, pulsesNum, 0, max_range_m],

    cmap="jet",

    origin='lower'

)

plt.xlabel("Slow Time (Pulse Index)")

plt.ylabel("Range (m)")

plt.ylim(min_dist_view, max_dist_view)

plt.colorbar(label="Magnitude")

```

```

plt.title("Range-Time Map (Clean Signal - Range in Meters)")

plt.show()

receivedSlice = receivedSignalWithNoise[0, :]

pulseTemplate = np.ones(int(Tp * Fs))
pulseTemplateFlipped = np.flip(pulseTemplate)

startTime = time.time()

rangeCompressed = np.zeros_like(receivedSignalWithNoise)

for n in range(pulsesNum):

    conv_res = np.convolve(receivedSignalWithNoise[n], pulseTemplateFlipped, mode='full')

    rangeCompressed[n, :] = conv_res[len(pulseTemplate) - 1: len(pulseTemplate) - 1 + samplesPerPulse]

matchedFilterOutputTimeDomain = np.sum(np.abs(rangeCompressed), axis=0)

endTime = time.time()

processingTimeMethod1 = (endTime - startTime) * 1000

startTime = time.time()

integrated_fft_acc = np.zeros(samplesPerPulse)

fftSize = 2 ** (int(np.log2(len(receivedSlice))) + 1)

pulseTemplateFft = np.fft.fft(pulseTemplate, fftSize)

for n in range(pulsesNum):

    receivedSliceFft = np.fft.fft(receivedSignalWithNoise[n], fftSize)

    fftProduct = receivedSliceFft * np.conj(pulseTemplateFft)

    fftOutputSingle = np.fft.ifft(fftProduct)

    integrated_fft_acc += np.abs(fftOutputSingle[:samplesPerPulse])

fftOutput = integrated_fft_acc

endTime = time.time()

processingTimeMethod2 = (endTime - startTime) * 1000

```

```

peakThreshold = 0.9 * np.max(matchedFilterOutputTimeDomain)

peaksTimeDomainIndices, _ = find_peaks(matchedFilterOutputTimeDomain, height=peakThreshold, distance=int(Tp * Fs) * 2)

peakTimesTimeDomain = fastTimeArray[peaksTimeDomainIndices]

detectedRangesTimeDomain = (peakTimesTimeDomain - startDelayTime) * c / 2


peaksFftIndices, _ = find_peaks(fftOutput, height=peakThreshold, distance=int(Tp * Fs) * 2)

peakTimesFft = fastTimeArray[peaksFftIndices]

detectedRangesFft = (peakTimesFft - startDelayTime) * c / 2


minTimeDomain = minTimeView

maxTimeDomain = maxTimeView

minFft = minTimeView

maxFft = maxTimeView


plt.figure(figsize=(12, 10))


plt.subplot(4, 1, 1)

plt.plot(fastTimeArray * 1e6, matchedFilterOutputTimeDomain, color='blue')

plt.title(f'Method 1: Time Domain Matched Filter ')

plt.xlabel('Time (us)')

plt.ylabel('Magnitude')

plt.grid(True)

plt.xlim(minTimeDomain, maxTimeDomain)


plt.subplot(4, 1, 2)

plt.plot(fastTimeArray * 1e6, fftOutput, color='green', linestyle='--')

plt.title(f'Method 2: FFT Based Processing ')

plt.xlabel('Time (us)')

plt.ylabel('Magnitude')

plt.grid(True)

plt.xlim(minFft, maxFft)

```

```

plt.subplot(4, 1, 3)

plt.plot(fastTimeArray * 1e6, matchedFilterOutputTimeDomain, color='blue', label='Signal')

if len(peaksTimeDomainIndices) > 0:

    plt.plot(peakTimesTimeDomain * 1e6, matchedFilterOutputTimeDomain[peaksTimeDomainIndices], marker='o',

             linestyle='none', fillstyle='none', color='red', markersize=12, label='Detected Peaks')

plt.axhline(peakThreshold, color='darkred', linestyle='--', label='Threshold')

plt.title('Peak Detection for Time Domain Method')

plt.xlabel('Time (us)')

plt.legend()

plt.grid(True)

plt.xlim(minTimeDomain, maxTimeDomain)


plt.subplot(4, 1, 4)

plt.plot(fastTimeArray * 1e6, fftOutput, color='green', label='Signal')

if len(peaksFftIndices) > 0:

    plt.plot(peakTimesFft * 1e6, fftOutput[peaksFftIndices], color='red', marker='o', fillstyle='none',

             linestyle='none', markersize=12, label='Detected Peaks')

plt.axhline(peakThreshold, color='darkred', linestyle='--', label='Threshold')

plt.title('Peak Detection for FFT Method')

plt.xlabel('Time (us)')

plt.legend()

plt.grid(True)

plt.xlim(minFft, maxFft)


plt.tight_layout()

plt.show()


print("=" * 60)


dopplerFftSize = pulsesNum

dopplerFrequencyAxis = np.fft.fftshift(np.fft.fftfreq(dopplerFftSize, d=PRI))

velocityAxis = (dopplerFrequencyAxis * c) / (2 * Fc)

```

```

detectedVelocitiesList = []

targetMatchIndices = []

plt.figure(figsize=(12, 6))

for detectionIdx, rangeBinIndex in enumerate(peaksTimeDomainIndices):

    slowTimeSignal = rangeCompressed[:, rangeBinIndex]

    dopplerSpectrum = np.fft.fftshift(np.fft.fft(slowTimeSignal, n=dopplerFftSize))

    spectrumMagnitude = np.abs(dopplerSpectrum)

    peakDopplerIndex = np.argmax(spectrumMagnitude)

    estimatedVelocity = velocityAxis[peakDopplerIndex]

    detectedVelocitiesList.append(estimatedVelocity)

    detectedRange = detectedRangesTimeDomain[detectionIdx]

    closestTrueTargetIndex = np.argmin(np.abs(np.array(targetsDistance) - detectedRange))

    targetMatchIndices.append(closestTrueTargetIndex)

    trueVelocity = targetsVelocity[closestTrueTargetIndex]

    movementDirection = "Approaching" if estimatedVelocity > 0 else "Receding"

    plt.subplot(1, len(peaksTimeDomainIndices), detectionIdx + 1)

    plt.plot(velocityAxis, spectrumMagnitude, 'b-', label='Doppler Spectrum')

    plt.axvline(x=estimatedVelocity, color='r', linestyle='--', label=f'Est: {estimatedVelocity:.1f} m/s')

    plt.axvline(x=trueVelocity, color='g', linestyle=':', linewidth=2, label=f'True: {trueVelocity} m/s')

    plt.title(f"Target at ~{detectedRange:.0f}m\n({movementDirection})")

    plt.xlabel("Velocity (m/s)")

    plt.ylabel("Magnitude")

    plt.grid(True)

    plt.legend()

```



```

plt.tight_layout()

plt.show()

rangeDopplerMap = np.fft.fft(rangeCompressed, n=pulsesNum, axis=0)

rangeDopplerMapShifted = np.fft.fftshift(rangeDopplerMap, axes=0)

rangeDopplerMapDb = 20 * np.log10(np.abs(rangeDopplerMapShifted) + 1e-10)

rangeAxis = (fastTimeArray - startDelayTime) * c / 2

plt.figure(figsize=(10, 8))

plt.imshow(rangeDopplerMapDb, aspect='auto', extent=[rangeAxis[0], rangeAxis[-1], velocityAxis[0], velocityAxis[-1]],
           origin='lower', cmap='viridis')

plt.colorbar(label='dB')

if len(detectedRangesTimeDomain) > 0:
    plt.plot(detectedRangesTimeDomain, detectedVelocitiesList, 'wx', markersize=10, markeredgewidth=2)

plt.xlabel("Range (m)")

plt.ylabel("Velocity (m/s)")

plt.title(f"Range-Doppler Map (SNR {SNR}dB)")

plt.xlim(min_dist_view, max_dist_view)

plt.show()

print(f"Detected Peak Times (Method 1): {peakTimesTimeDomain}")

print(f"Detected Peak Times (Method 2): {peakTimesFft}")

print(f"Detected Ranges (Method 1): {detectedRangesTimeDomain}")

print(f"Detected Ranges (Method 2): {detectedRangesFft}")

print(f"\nProcessing Time (Method 1 - Time Domain): {processingTimeMethod1:.3f} ms")

print(f"Processing Time (Method 2 - FFT Domain): {processingTimeMethod2:.3f} ms")

print("\n--- VELOCITY COMPARISON ---")

print(f"{'Target':<10}{'True Velocity (m/s)':<25}{'Estimated Velocity (m/s)':<30}{'Error (m/s)':<20}{'Direction':<15}")

for i in range(len(detectedVelocitiesList)):

```

```
detectedR = detectedRangesTimeDomain[i]

closestTrueIdx = np.argmin(np.abs(np.array(targetsDistance) - detectedR))

rightVelocity = targetsVelocity[closestTrueIdx]

estVelocity = detectedVelocitiesList[i]

velocityError = abs(estVelocity - rightVelocity)

direction = "Approaching" if estVelocity > 0 else "Receding"

print(f"{i + 1:<10}{rightVelocity:<25.2f}{estVelocity:<30.2f}{velocityError:<20.2f}{direction:<15}")
```